PHASE – 2 of Javascript

Javascript is:

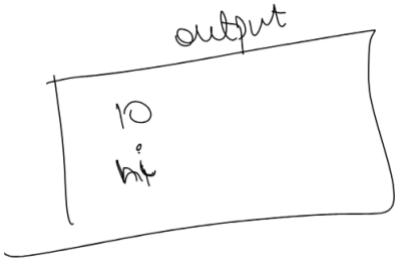
- Single threaded
- Weakly typed
- Dynamically typed
- Interpreted programming language

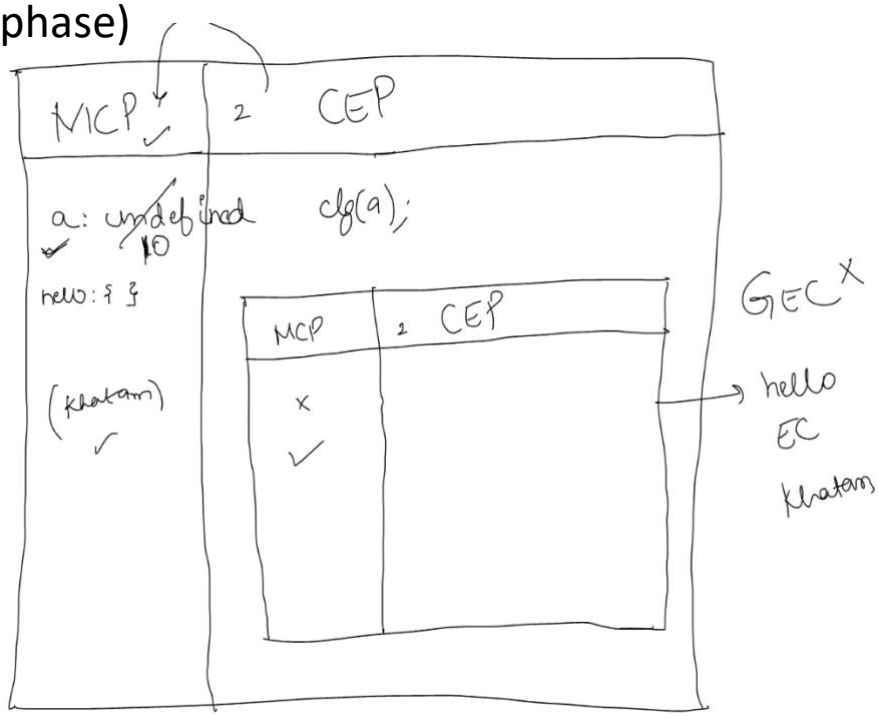
Shortest code of javascript is empty JS file.

Whenever a javascript code is done, a global execution context(GEC) is created inside that GEC we have two phases

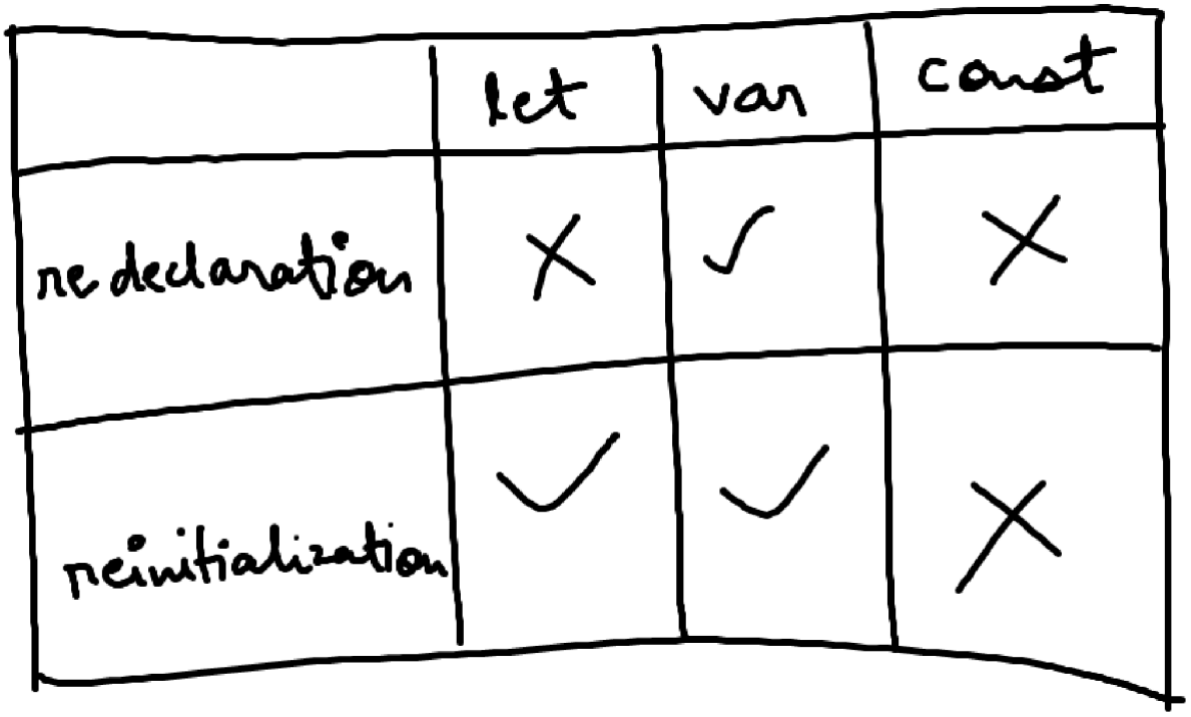
1. MCP(Memory creation phase)
2. CEP(Code execution phase)

```
let a = 10;
console.log(a);
function hello(){
    console.log("hi")
}
hello();
```





Whenever a function is called a new execution context is created with the name of the function which also has two phases(same as above).



Ex :

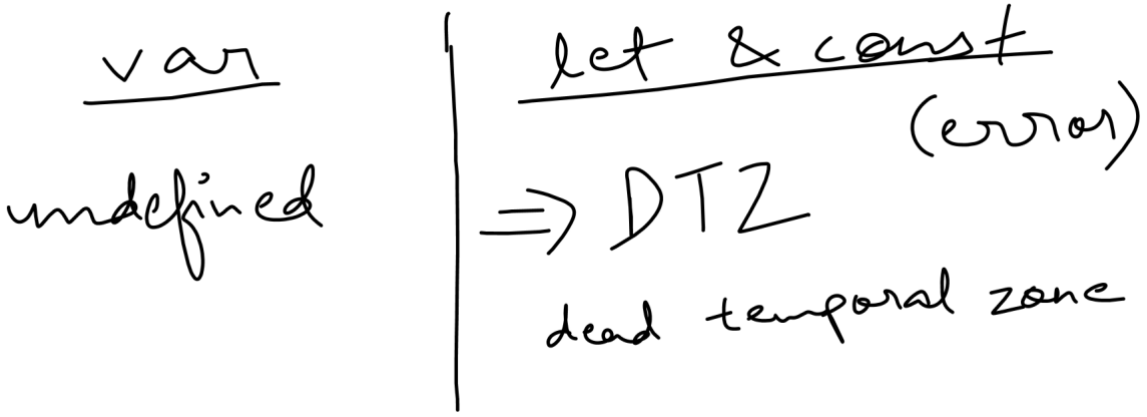
- Let a = 10;
  a = 100;
  clg(a);
  a = “Babua”;
  clg(a);
- var chup = “red”;
  chup = 10;
  var chup = “green”;
- const a = 10;
  clg(a);
  a = 20;
- const a; → undefined
  a = 0;
  clg(a);

In case of const, variable must be initialized in the same line where this variable is declared, otherwise error will be generated.

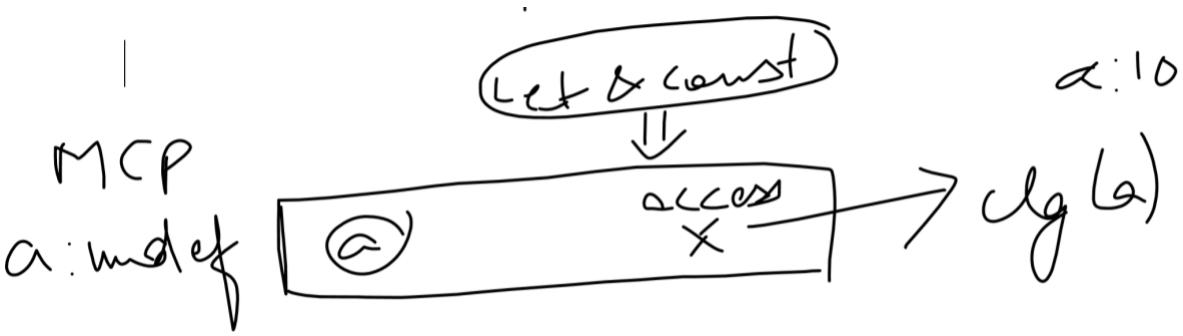
Hoisting -> accessing a variable or a function before its declaration is called hoisting.

let, var, const ….all are hoists.

Hoisting in different in var and in let and const.



Dead temporal zone –

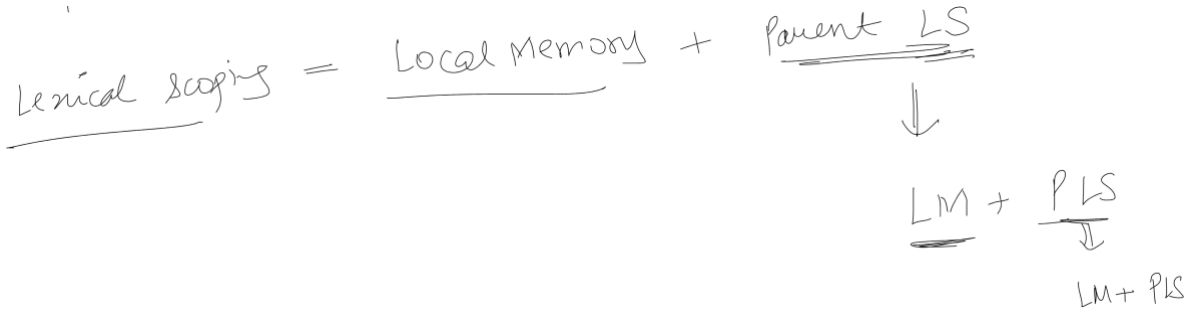


Let and const se declaration par CEP DTZ se access karega jiski wajah se error generate hoga…while in case of var…CEP will take the value directly from MCP.

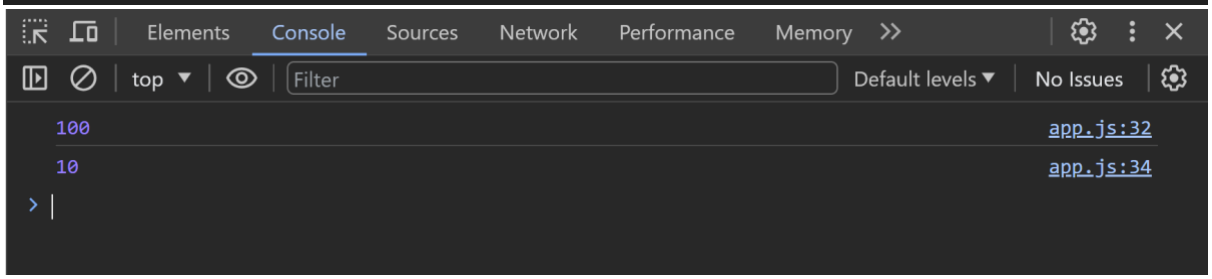
- It is an imaginary zone b/w MCP and CEP.
- It gets activated in case of let and const.
- CEP checks whether variable is initilised or not.

Scoping

Lexical scoping – Lexical scope is defined as local memory(MCP) + Parent's Lexical Scope(which means parent's local memory and its parent's lexical scope).




```
let b = 10;
function fun1(){
    let c = 100;
    function fun2(){
        console.log(c);
        function fun3(){
            console.log(b)
        }
        fun3();
    }
    fun2();
}
fun1();
```



Scope – from where a variable can be accessed.

- Whenever a JS code is run a global execution context(GEC) is created, along with it a global object is also created.
- In our case, i.e., since we are using the browser the global object is window.
- In case of scoping, let and var are considered to be having two different scopes.
    - var –
        - global scope
        - functional scope.
    - let/const –
        - local scope/ script scope
        - block scope.

Ques – Why in JS functions are called first class citizens?

OR

What are first class functions in JS?

Ans – In JavaScript, whenever you are able to assign a variable to a function then that function will be called as first-class function and this concept of assigning a variable to a value function is called first class citizenship.

Ques – What are higher order functions?

Ans – There are two possible situations for a higher order function:

- When you have two functions (function 1, function 2) and any one of these function (assuming

function 2) is being sent as an argument to the other function (function 1) then fun 1 is called higher order function.

```
function fun1(fn){
    console.log('fun1');
    fn();
}

function fun2(){
    console.log('fun2');
}

fun1(fun2); //hof
```

```
fun1
fun2
Live reload enabled.
```

- When you have two functions (function 1, function 2) if one of the function (function 1) can return the other function (function 2) from itself then function 1 is called higher order function.

```
function fun1(){
    console.log("inside fun 1");
    function fun2(){
        console.log("inside fun 2");
    }
    return fun2;
}

let returnedValue = fun1();
returnedValue();
```

```
inside fun 1
inside fun 2
```

Note: When nothing is returned from a function, by default undefined is returned.

Ques – What is call back function?

Ans – A call back function is basically when you pass it as an argument to some higher order function and that passed function is being called inside that higher order function, this

calling criterion is must, then only it will called as call back function.

Closure –

Whenever you return a function that function is never returned alone, it always takes along the lexical environment with it, so that if in future we ever want to run that returned function then it should not throw an error.

Use case of closure:

Before the class syntax was introduced, we had no way to privatize our variables or functions, so in that case we use closures so that we can privatize our variables.

Example –

```
function counter(){
    let count =0;
    return {
        getCount:function(){console.log(count)},
        increment:function(){count++;}
    }
}

counter().getCount();
```

In above example we cannot directly use the count variable but we can get count using function and hence, this is how we can privatize data.

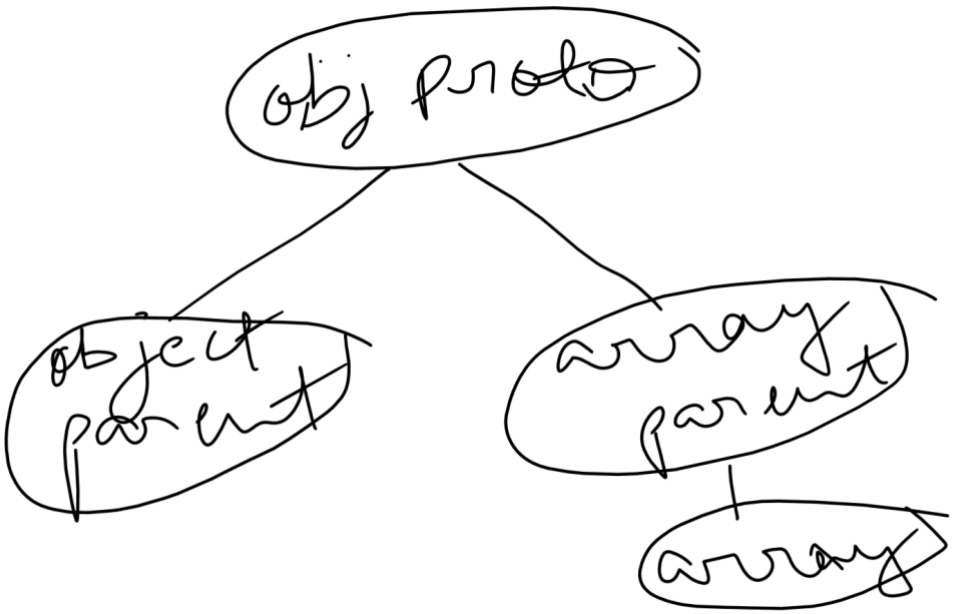
Prototype

To check parent of an object we use property “dender proto” (__proto__)

Important –

Obj.prototype == obj.__proto__

```
> obj.prototype.add = function sayhello(){console.log('hello world')}
✖ ▶Uncaught TypeError: Cannot set properties of undefined (setting 'add')
    at <anonymous>:1:19
> obj __proto__
✖ Uncaught SyntaxError: Unexpected identifier '__proto__'
> obj.get__proto__
‹ undefined
> obj.__proto
‹ undefined
> obj.__proto__
‹ ▼{constructor: ƒ, __defineGetter__: ƒ, __defineSetter__: ƒ, hasOwnProperty: ƒ, __lookupGetter__: ƒ, …}
    ▶constructor: ƒ Object()
    ▶hasOwnProperty: ƒ hasOwnProperty()
    ▶isPrototypeOf: ƒ isPrototypeOf()
    ▶propertyIsEnumerable: ƒ propertyIsEnumerable()
    ▶toLocaleString: ƒ toLocaleString()
    ▶toString: ƒ toString()
    ▶valueOf: ƒ valueOf()
    ▶__defineGetter__: ƒ __defineGetter__()
    ▶__defineSetter__: ƒ __defineSetter__()
    ▶__lookupGetter__: ƒ __lookupGetter__()
    ▶__lookupSetter__: ƒ __lookupSetter__()
      __proto__: (...)
    ▶get __proto__: ƒ __proto__()
    ▶set __proto__: ƒ __proto__()
> obj.__proto__.add
‹ undefined
> obj.__proto__.add(function wayhello(){console.log('hello')})
✖ ▶Uncaught TypeError: obj.__proto__.add is not a function
    at <anonymous>:1:15
›
```



Hence, above diagram proves that everything in JS is an object.

Constructor –

Constructor is used to create an object.

Always start a constructor with a capital letter.

```
function Sam(){
    this.naam = "kaju";
    this.umar = 2;
}
let out = new Sam();
console.log(out);
```

```
▼ Sam {naam: 'kaju', umar: 2} i
    naam: "kaju"
    umar: 2
  ▼ [[Prototype]]: Object
    ▶ constructor: ƒ Sam()
    ▶ [[Prototype]]: Object
›
```

Here, Sam is a constructor.

This keyword is used to create properties of the constructor.

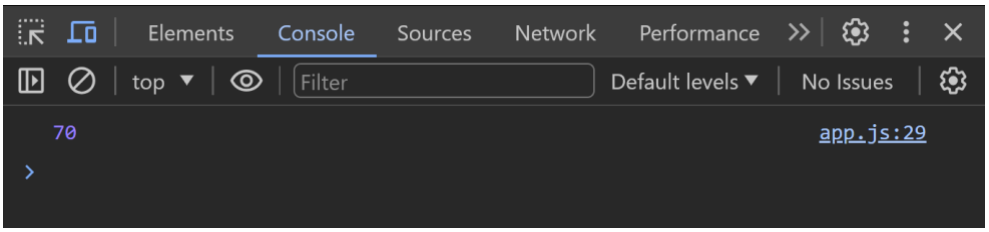
The prototype of the object created from a constructor function is the prototype of that function itself.

Arrow Functions –

For our convenience, Samarth sir has categorized arrow function declaration in three ways :-

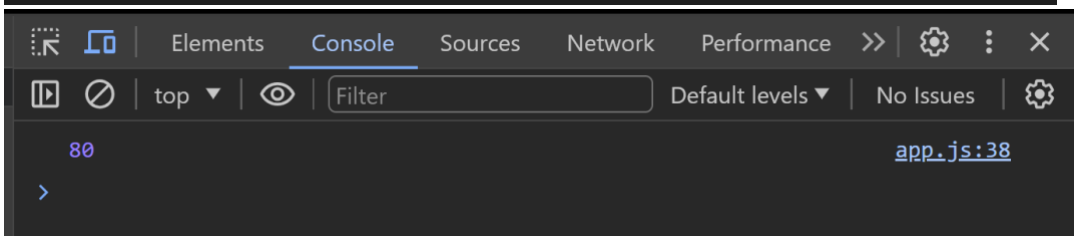
- **Way 1 –**

```
let sum1 = (a, b, c) => {
    return a + b + c;
}

let ans1 = sum1(10, 20, 40);
console.log(ans1);
```



➢ **Way 2 –**

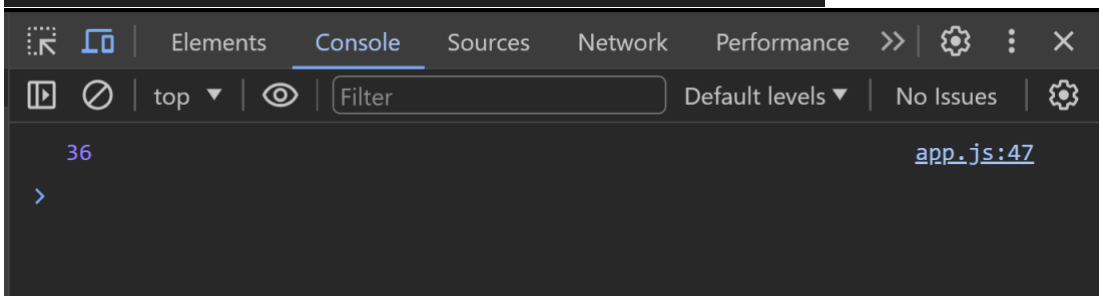
```
let sum2 = (a, b, c) => a + b + c;

let ans2 = sum2(20, 20, 40);
console.log(ans2);
```



In this way, if there is a single line in function, then the arrow function will return it.

➢ **Way 3 –**

```
// let sqr = (a) => a * a;
let sqr = a => a * a;

let ans3 = sqr(6);
console.log(ans3);
```



We can ignore the parentheses as well in case if there is only one argument.

‘This’ Keyword

‘This’ depends on how it is being called upon.

There are 5 types of this keyword :-

- Object/Method calling
- Direct function calling
- Constructor calling
- Indirect calling
- Arrow function calling

1) Object/Method calling –

If it is being called by an object then it will return the object.

```
let obj = {
    a: 10,
    b: 200,
    fn: function () {
        console.log(this);
    }
}

obj.fn();
```

```
▼ {a: 10, b: 200, fn: ƒ}
    a: 10
    b: 200
  ▶ fn: ƒ ()
  ▶ [[Prototype]]: Object
```

2) Direct calling –

If it is directly called by a function then it returns a window object.

```
function sam() {
    console.log(this);
}

sam();  //direct calling
```

```
                                                    app.js:19
▶ Window {window: Window, self: Window, document: document, name: '', loc
ation: Location, …}
```

3) Constructor calling –

In this case, a newly created object will be returned.

```
function Sam(){
    this.name = "sam";
}

let obj1 = new Sam(); // newly created object
console.log(obj1);
```

```
▶ Sam {name: 'sam'}
>
```

4) Arrow calling –

It will also return a window object.

```
let obj5 = {
    a: 20,
    fn: () => {
        console.log(this);
    }
}

obj5.fn(); // window
```

```
                                                    app.js:94
▶ Window {window: Window, self: Window, document: document, name: '', Loc
  ation: Location, …}
>
```